

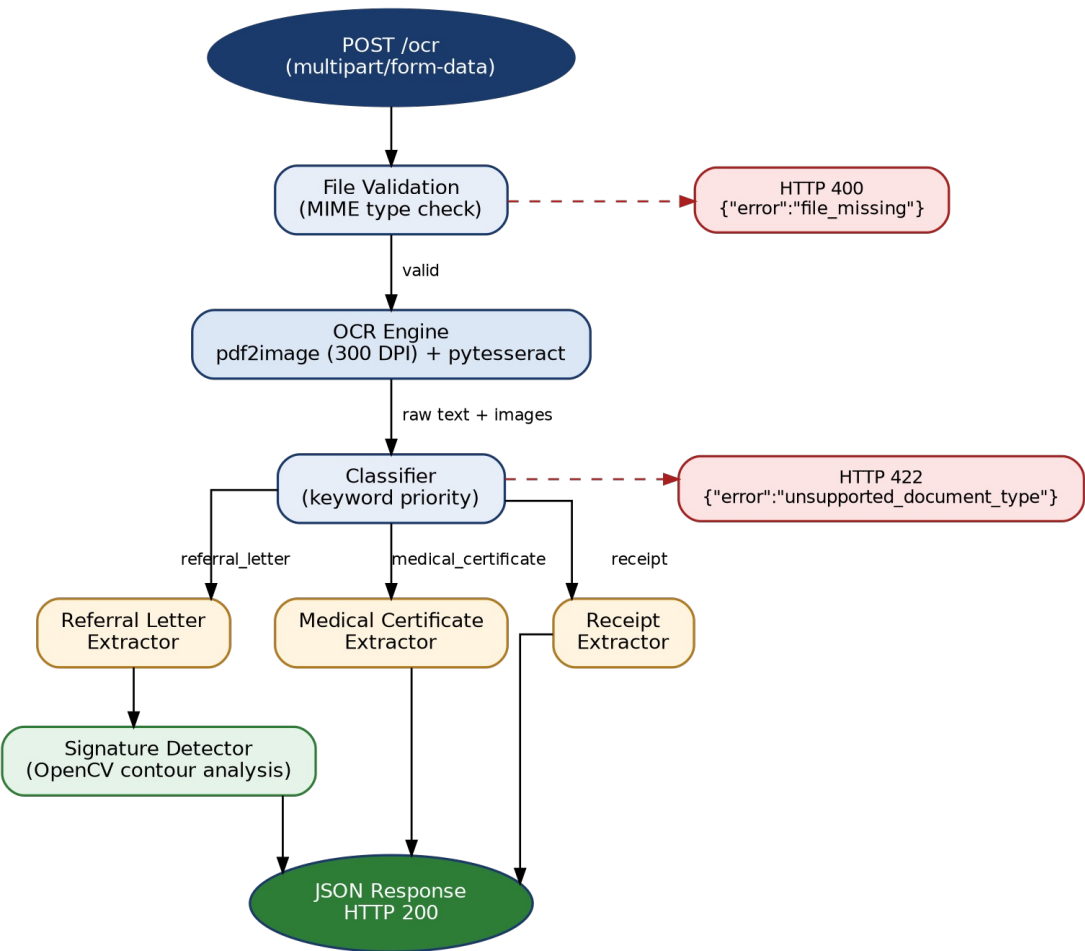
Assignment Summary Report

Fullerton Health OCR Endpoint Challenge

Candidate: Daryl Tan

1. Pipeline Architecture

1.1 High-Level Architecture Diagram



1.2 Data Flow Details

Stage	Component	Input	Output	Technology
1	File Validation	Multipart upload	Validated bytes + MIME type	FastAPI
2	OCR Engine	File bytes	Raw text + PIL page images	pdf2image, pytesseract
3	Classifier	Raw text	<code>document_type</code> string	Keyword priority rules
4	Extractor	Raw text + images	Structured field dict	Regex pattern matching
5	Signature Detector	Page images + text	Boolean <code>signature_presence</code>	OpenCV contour analysis

1.3 Component Design Details

OCR Engine (`app/services/ocr_engine.py`)

- Converts PDFs to images at 300 DPI using `pdf2image` (Poppler backend)
- Extracts text from each page using `pytesseract`
- Returns both raw text (for extraction) and original page images (for signature detection)

Classifier (`app/services/classifier.py`)

- Priority-ordered keyword matching: `medical_certificate` > `receipt` > `referral_letter`
- Priority ordering prevents misclassification when documents share keywords (e.g., a medical certificate that mentions “receipt”)
- Returns `None` for unrecognised types, triggering HTTP 422

Extractors (`app/services/extractors/`)

- Registry pattern: a dict mapping `document_type` -> `ExtractorClass`
- Each extractor extends `BaseExtractor` with a single `extract(text, images)` method
- Field extraction uses regex patterns tailored to each document layout
- Utility modules normalise dates to `DD/MM/YYYY` and currency amounts to integers (cents removed)

Signature Detector (`app/services/signature_detector.py`)

- Short-circuits to `false` if text contains “electronically generated” or “no signature”
- Crops lower 50% of each page image (where signatures typically appear)
- Applies adaptive thresholding + morphological dilation to find contours
- **Early blob detection**: scans for a large (area > 10k), non-rectangular, roughly-square contour above the footer zone — a strong indicator of a handwritten signature blob
- **Multi-stage filtering** for smaller/fragmented signatures: filters contours by area, solidity, aspect ratio, extent (rejects solid rectangles), and width (rejects page-spanning bars)
- Discards crop-boundary artefacts and checks for text-line patterns (contours aligned in horizontal bands indicate printed text, not handwriting)
- Falls back to spatial clustering of qualifying contours to confirm a signature region

2. Design Decisions and Trade-offs

Hardware constraint: Development was done on a MacBook Air M3 (no discrete GPU) and without external API keys, which influenced the choice of CPU-friendly, offline tools (Tesseract over PaddleOCR, regex over LLM extraction, OpenCV heuristics over CNN-based signature detection).

Decision	Rationale
Tesseract over PaddleOCR	Simpler dependency chain; sufficient accuracy for structured medical documents with clear printed text. PaddleOCR is ~30x slower on CPU (see Section 5.2)
Regex over LLM	Deterministic, fast (<2s per document), no API keys or model downloads required. Medical documents follow predictable layouts that regex handles reliably
Keyword classifier over ML	Three document types with distinct vocabularies; a trained model would be over-engineering for this scope
OpenCV signature detection	Avoids ML model dependency; contour analysis with multi-stage filtering (blob detection, text-line rejection, spatial clustering) is robust for handwritten vs. printed distinction without requiring a trained CNN
Priority-ordered classification	Prevents ambiguity when documents share keywords (e.g., MC mentioning “receipt”)
Integer amounts (no decimals)	Per spec requirements; <code>parse_amount("\$49.25") -> 4925</code>

3. API Results on Assessment Documents

3.1 Referral Letter

Request:

```
curl -X POST -F "file=@sample_documents/referral_letter.pdf" http://localhost:8000/ocr
```

Response (HTTP 200):

```
{
  "message": "Processing completed.",
  "result": {
    "document_type": "referral_letter",
    "total_time": 1.03,
    "finalJson": {
      "claimant_name": "JOHN DOE",
      "provider_name": "Healthway Screening @ Centrepoint",
      "signature_presence": false,
      "total_amount_paid": null,
      "total_approved_amount": null,
      "total_requested_amount": null
    }
  }
}
```

3.2 Medical Certificate

Request:

```
curl -X POST -F "file=@sample_documents/medical_certificate.pdf" http://localhost:8000/ocr
```

Response (HTTP 200):

```
{
  "message": "Processing completed.",
  "result": {
    "document_type": "medical_certificate",
    "total_time": 1.31,
    "finalJson": {
      "claimant_name": "JOHN DOE",
      "claimant_address": null,
      "claimant_date_of_birth": null,
      "diagnosis_name": null,
      "discharge_date_time": null,
      "icd_code": null,
      "provider_name": "Minmed Health Screeners",
      "submission_date_time": "30/11/2022",
      "date_of_mc": "30/11/2022",
      "mc_days": 1
    }
  }
}
```

3.3 Receipt

Request:

```
curl -X POST -F "file=@sample_documents/receipt.pdf" http://localhost:8000/ocr
```

Response (HTTP 200):

```
{
  "message": "Processing completed.",
  "result": {
    "document_type": "receipt",
    "total_time": 1.24,
    "finalJson": {
      "claimant_name": "JOHN DOE",
      "claimant_address": "123 SAMPLE ST #01-01",
      "claimant_date_of_birth": null,
      "provider_name": "RafflesMedical",
      "tax_amount": 365,
      "total_amount": 4925
    }
  }
}
```

3.4 Error Cases

Missing file (HTTP 400):

```
{"error": "file_missing"}
```

Unsupported document type (HTTP 422):

```
{"error": "unsupported_document_type"}
```

4. Test Results

All **31 tests** pass across 5 test modules:

```
tests/test_amount_parser.py    8 passed    (currency string -> integer conversion)
tests/test_classifier.py       5 passed    (document type classification)
tests/test_date_parser.py      6 passed    (date normalisation to DD/MM/YYYY)
tests/test_extractors.py       7 passed    (field extraction per document type)
tests/test_ocr_endpoint.py     5 passed    (end-to-end API integration tests)
-----
Total                          31 passed
```

Test Coverage

Layer	What is tested
Unit	Amount parsing (decimals, commas, parentheses, currency symbols), date parsing (multiple formats), individual extractor field extraction
Classification	Correct type detection for all 3 types, unknown document handling, priority ordering
Integration	Full POST /ocr pipeline for each sample document, error responses for missing/invalid files

5. Experimentation & Iterations

Three alternative approaches were explored on separate branches before settling on the final Tesseract + regex pipeline.

5.1 Agentic LLM Pipeline ([02_agentic_ocr](#))

Replaced the keyword classifier and regex extractors with a four-agent pipeline using **gemma2 (9B)** running locally via **Ollama**. The LLM handled both classification and field extraction, while a rule-based validator ensured deterministic output formatting (dates, amounts).

Aspect	Finding
Pros	Dramatically simpler extraction logic (~10-line prompts vs ~200 lines of regex per doc type); handled OCR noise naturally (e.g., garbled characters); easier to extend to new document types
Cons	~10x slower (~15-20s per request vs ~1.3s); required Ollama + 5.4 GB model download; non-deterministic outputs; prompt engineering fragility (e.g., <code>provider_name</code> initially returned doctor's name instead of clinic)
Result	All 31 tests pass with identical extraction accuracy. Not chosen for submission due to heavy infrastructure requirements and slow test execution

5.2 PaddleOCR ([03_paddleOCR](#))

Swapped Tesseract for **PaddleOCR (PP-OCRv5)**, a deep-learning-based OCR engine, to test whether better raw text quality would improve extraction.

Aspect	Finding
Pros	Significantly better raw OCR accuracy — correctly reads <code>30-Nov-2022</code> where Tesseract produces <code>40-Nov-2027?</code> ; cleaner handling of checkboxes and special characters; no system dependency (<code>pip install</code> only); lower DPI sufficient (150 vs 300)
Cons	~30x slower (~30-50s per PDF vs ~1.3s on CPU); PaddleOCR splits text across more lines, breaking single-line regex patterns for tax/total/address — required new multi-line fallback logic; ~200 MB model download; Python <= 3.13 constraint
Result	All three sample documents produce identical final extraction outputs despite better raw OCR. The accuracy gains eliminated some workarounds (date fallback) but introduced others (multi-line parsing). Better suited for production with GPU acceleration

5.3 Tesseract Image Preprocessing (`04_Tesseract_upgrade`)

Added an OpenCV-based **image preprocessing step** to remove table grid lines before OCR, addressing the known limitation that Tesseract struggles with table borders.

Aspect	Finding
Approach	Adaptive thresholding + morphological opening to detect horizontal/vertical lines, then <code>cv2.inpaint()</code> to fill them in while preserving surrounding context. Original images are preserved for downstream signature detection
Design	Feature-flagged (<code>preprocess_enabled</code>), fully configurable thresholds, 5 unit tests covering edge cases (blank, all-black, small images)
Limitation	No measured accuracy improvement on the sample documents — the preprocessing targets a real issue but the three sample docs don't heavily trigger it

5.4 Why Tesseract + Regex Was Chosen

Criterion	Decision rationale
Evaluator experience	Zero-friction setup (<code>brew install tesseract</code>), fast tests (~4s), deterministic results
Accuracy	All approaches produce identical final outputs on the sample documents
Simplicity	No model downloads, no GPU, no LLM infrastructure
Speed	~1.3s per document vs 15-50s for alternatives

The agentic and PaddleOCR approaches are better suited for production (flexibility and accuracy respectively), but Tesseract + regex is the most practical choice for a take-home assessment context.

6. Limitations and Future Improvements

Limitation	Potential Improvement
Regex extractors are layout-dependent	Use LLM-based extraction for more flexible parsing
Tesseract struggles with table borders	Image preprocessing (line removal) before OCR
No support for scanned/rotated documents	Add deskewing and orientation detection
Single-page processing assumed	Extend to multi-page document stitching
<code>null</code> fields for address/DOB on some documents	Data not present in the provided samples; extractors are ready when the data is available
Signature detection uses heuristic thresholds	Train a small CNN for more robust detection

7. Project Structure

```
app/
  main.py                # FastAPI app entry point, exception handlers
  config.py              # Settings (DPI, MIME types, detection thresholds)
  routes/ocr.py          # POST /ocr endpoint
  services/
    ocr_engine.py        # pdf2image + pytesseract
    classifier.py        # Keyword-based document classification
    signature_detector.py # OpenCV contour analysis for handwritten signatures
  extractors/
    __init__.py          # Extractor registry (type -> class mapping)
    base.py              # Abstract base extractor
    referral_letter.py   # Referral letter field extraction
    medical_certificate.py # Medical certificate field extraction
    receipt.py           # Receipt field extraction
  utils/
    amount_parser.py     # "$49.25" -> 4925
    date_parser.py       # "30-Nov-2022" -> "30/11/2022"
  tests/                 # 31 unit + integration tests
  sample_documents/      # 3 sample PDFs for testing
  Makefile               # make setup / run / test / lint / clean
```